Master MIAGE – Architecture Microservices

Projet Microservices

Application de Gestion de Bibliothèque

Étudiantes

Sirine BOUGHANEM
Chaimaa ARSALANE

Encadrant

Benoît Charroux

Dépôt Git : `https://github.com/Sirinamarwa/kubernetes-projet`
Images Docker Hub :
`sirinamarwa/books-service:latest`
`chaimaaaa/user-loans-service:latest`
Technologies : Node.js • PostgreSQL • Docker • Kubernetes • React/Vite

Année universitaire 2025 – 2026

Table des matières

1 Introduction **2**

2 Architecture générale **2**

3 Service A : Books Service **3**
- 3.1 Fonctionnalités . . . 3
- 3.2 Routes . . . 3
- 3.3 Kubernetes (k8s/base) . . . 3

4 Service B : User Loans Service **3**
- 4.1 Fonctionnalités . . . 4
- 4.2 Routes . . . 4
- 4.3 Codes de réponse métier . . . 4
- 4.4 Kubernetes (k8s/loans) . . . 4

5 Base de données **5**
- 5.1 Table `books` . . . 5
- 5.2 Table `loans` . . . 5

6 Communication inter-services **6**
- 6.1 Flux d'un emprunt (`POST /loans`) . . . 6
- 6.2 Flux d'un retour (`PUT /loans/:id/return`) . . . 6

7 Docker et Docker Hub **6**
- 7.1 Books Service . . . 6
- 7.2 Loans Service . . . 6

8 Lancement local avec Docker Compose **7**

9 Frontend React/Vite **7**

10 Conclusion **7**

1. Introduction

Ce projet a pour objectif de concevoir et déployer une application de gestion de bibliothèque en suivant une architecture microservices. Chaque fonctionnalité est encapsulée dans un service indépendant, déployable et scalable de manière autonome.

L'architecture microservices a été choisie pour plusieurs raisons :

— **Séparation des responsabilités** : le catalogue de livres et la gestion des emprunts sont deux domaines métier distincts qui évoluent indépendamment ;

— **Résilience** : la panne d'un service n'affecte pas l'autre ;

— **Scalabilité** : chaque service peut être mis à l'échelle indépendamment selon la charge.

Objectifs techniques du projet :

— développer deux microservices REST en Node.js/Express ;

— mettre en place une communication synchrone inter-services via HTTP ;

— persister les données avec PostgreSQL (une base par service) ;

— conteneuriser chaque service avec Docker et publier les images sur Docker Hub ;

— orchestrer l'ensemble avec Kubernetes (Deployments, Services, Ingress, Secrets, RBAC, PVC) ;

— proposer une interface web React de démonstration.

2. Architecture générale

L'application est décomposée en trois couches principales :

Composant	Rôle
`books-service`	Catalogue des livres – CRUD + disponibilité
`user-loans-service`	Gestion des emprunts – orchestration + communication
`PostgreSQL (books)`	Persistance du catalogue
`PostgreSQL (loans)`	Persistance des emprunts
`frontend`	Interface React/Vite – catalogue + emprunts
`k8s/base`	Manifests Kubernetes du catalogue
`k8s/loans`	Manifests Kubernetes des emprunts (RBAC, Secrets, PVC)
`docker-compose.yml`	Orchestration locale complète

Flux de communication :

1. Consultation du catalogue :
Utilisateur → Frontend → `books-service` → PostgreSQL (books)

2. Emprunt d'un livre :
Utilisateur → Frontend → `user-loans-service` → PostgreSQL (loans)
$HTTPinterne$ `books-service` → PostgreSQL (books)

3. Retour d'un livre :
Utilisateur → Frontend → `user-loans-service` $HTTPinterne$ `books-service`

mise à jour statut + PostgreSQL (books)

3. Service A : Books Service

`books-service` est développé en Node.js avec Express et PostgreSQL. Il expose une API REST pour la gestion complète du catalogue de la bibliothèque.

3.1. Fonctionnalités

— lister tous les livres avec filtres (disponibilité, recherche par titre ou auteur) ;
— récupérer un livre par identifiant ;
— créer, modifier et supprimer un livre ;
— exposer un champ métier `status` : `AVAILABLE` ou `BORROWED` ;
— permettre à `user-loans-service` de modifier la disponibilité via des routes PATCH.

3.2. Routes

Route	Description
`GET /books`	lister tous les livres
`GET /books?available=true`	filtrer les livres disponibles
`GET /books?search=...`	rechercher par titre ou auteur
`GET /books/:id`	récupérer un livre
`POST /books`	ajouter un livre
`PUT /books/:id`	modifier un livre
`PATCH /books/:id/availability`	changer la disponibilité
`PATCH /books/:id/borrow`	marquer comme emprunté (appel inter-service)
`PATCH /books/:id/return`	marquer comme disponible (appel inter-service)
`DELETE /books/:id`	supprimer un livre

3.3. Kubernetes (k8s/base)

L'ensemble des manifests Kubernetes pour `books-service` est configuré dans `k8s/base` : namespace `library-app`, ConfigMap, Secret, StatefulSet PostgreSQL, Deployment, Service et Ingress. Application via Kustomize :

```
kubectl apply -k k8s/base
```

4. Service B : User Loans Service

`user-loans-service` est développé en Node.js avec Express et PostgreSQL. Il prend en charge la logique métier des emprunts et orchestre la communication avec `books-service` pour maintenir la cohérence des données entre les deux services.

4.1. Fonctionnalités

— créer un emprunt après vérification que le livre existe et est disponible ;
— refuser automatiquement un emprunt si le livre est déjà prêté (HTTP 409) ;
— lister tous les emprunts actifs et historiques ;
— lister les emprunts d'un utilisateur spécifique par son identifiant ;
— enregistrer le retour d'un livre et remettre sa disponibilité à jour dans `books-service` ;
— supprimer un emprunt ;
— health check via `GET /health` pour la supervision Kubernetes.

4.2. Routes

Route	Description
`GET /health`	vérification de l'état du service
`GET /loans`	lister tous les emprunts
`GET /loans/user/:userId`	emprunts d'un utilisateur
`POST /loans`	créer un emprunt
`PUT /loans/:id/return`	retourner un livre
`DELETE /loans/:id`	supprimer un emprunt

4.3. Codes de réponse métier

Code HTTP	Signification
201	emprunt créé avec succès
200	opération réussie (retour, liste)
404	livre introuvable dans le catalogue
409	livre déjà emprunté – conflit métier
500	erreur interne

4.4. Kubernetes (k8s/loans)

L'ensemble des manifests Kubernetes pour `user-loans-service` est configuré dans `k8s/loans` :

— `deployment.yaml` : Deployment Node.js avec injection des variables d'environnement ;
— `statefulset-postgres.yaml` : StatefulSet PostgreSQL pour la haute disponibilité des données ;
— `pvc.yaml` : PersistentVolumeClaim de 1 Gi – les données survivent au redémarrage des pods ;
— `service.yaml` : Service ClusterIP pour l'exposition interne dans le cluster ;
— `ingress.yaml` : Ingress nginx qui route `/loans` vers le service ;
— `secret.yaml` : Secrets Kubernetes pour les credentials PostgreSQL ;
— `rbac.yaml` : contrôle d'accès basé sur les rôles.

Secrets Kubernetes

Les credentials de la base de données (hôte, port, nom, utilisateur, mot de passe) sont stockés dans un `Secret` Kubernetes encodé en base64. Ils sont ensuite injectés en variables d'environnement dans le container via `envFrom`. Aucun secret n'apparaît en clair dans les fichiers de configuration ou dans le code source.

RBAC – Principe du moindre privilège

Un contrôle d'accès granulaire est mis en place via trois ressources Kubernetes :

- `ServiceAccount loans-service-account` : identité dédiée au pod, séparée du compte par défaut ;
- `Role` : liste explicite des permissions minimales nécessaires au service ;
- `RoleBinding` : association du Role au ServiceAccount.

Ce mécanisme empêche le pod d'accéder à des ressources Kubernetes non autorisées, conformément aux bonnes pratiques de sécurité en environnement de production.

Application :

```
kubectl apply -f k8s/loans/
```

5. Base de données

Chaque service dispose de sa propre instance PostgreSQL, déployée comme StatefulSet Kubernetes avec un PersistentVolumeClaim. Cette isolation garantit l'indépendance des services.

5.1. Table `books`

```
CREATE TABLE books (
  id        VARCHAR(100) PRIMARY KEY,
  title     VARCHAR(255) NOT NULL,
  author    VARCHAR(255),
  available BOOLEAN DEFAULT true
);
```

5.2. Table `loans`

```
CREATE TABLE loans (
  id          SERIAL PRIMARY KEY,
  user_id     VARCHAR(100)  NOT NULL,
  book_id     VARCHAR(100)  NOT NULL,
  loan_date   TIMESTAMP     DEFAULT NOW(),
  return_date TIMESTAMP,
  status      VARCHAR(20)   DEFAULT 'active'
);
```

Le champ `status` prend les valeurs `active` ou `returned`. Le champ `return_date` est `NULL` tant que le livre n'est pas retourné.

6. Communication inter-services

La communication inter-services permet aux deux microservices de rester cohérents sans partager de base de données.

`user-loans-service` appelle `books-service` via l'URL interne Kubernetes :

```
BOOK_CATALOG_URL=http://books-service.library-app.svc.cluster.local:3001
```

Ce DNS interne est résolu par le *kube-dns* de Kubernetes : le service `books-service` du namespace `library-app` est contacté directement, sans passer par un port externe ni par l'Ingress.

6.1. Flux d'un emprunt (`POST /loans`)

1. `GET /books/:id` — le service vérifie que le livre existe dans le catalogue ;
2. si le livre est introuvable, retourne `404` sans créer d'emprunt ;
3. si le champ `available` est `false`, retourne `409` (conflit métier) ;
4. `PATCH /books/:id/borrow` — le livre est marqué comme emprunté dans `books-service` ;
5. l'emprunt est enregistré en base PostgreSQL avec le statut `active`.

6.2. Flux d'un retour (`PUT /loans/:id/return`)

1. l'emprunt est mis à jour : `status = returned`, `return_date = NOW()` ;
2. `PATCH /books/:id/return` — le livre est remis en disponibilité dans `books-service`.

> **Garantie de cohérence :** si l'appel à `books-service` échoue, l'emprunt n'est pas créé (ou le retour n'est pas enregistré). Les deux opérations sont toujours synchronisées.

7. Docker et Docker Hub

Chaque service est conteneurisé indépendamment avec son propre `Dockerfile`. Les images sont publiées sur Docker Hub et référencées dans les manifests Kubernetes.

7.1. Books Service

```
docker build -t books-service-local ./services/books-service
docker tag books-service-local sirinamarwa/books-service:latest
docker push sirinamarwa/books-service:latest
```

7.2. Loans Service

```
docker build -t chaimaaaa/user-loans-service:latest
↪  ./services/loans-service
docker push chaimaaaa/user-loans-service:latest
```

8. Lancement local avec Docker Compose

Un fichier `docker-compose.yml` commun permet de lancer l'intégralité de l'application en une seule commande :

```
docker compose up --build
```

Services exposés en local :

Service	URL
`books-service`	`http://localhost:3001`
`user-loans-service`	`http://localhost:3002`
`frontend`	`http://localhost:5173`

Tests rapides :

```
curl http://localhost:3001/books
curl http://localhost:3002/health
curl -X POST http://localhost:3002/loans \
  -H "Content-Type: application/json" \
  -d '{"user_id":"u1","book_id":"book-1"}'
```

9. Frontend React/Vite

L'interface utilisateur est réalisée en React/Vite et intègre les deux microservices.

Fonctionnalités de l'interface :

- catalogue de livres avec recherche et filtre par disponibilité ;
- ajout d'un livre via une modale ;
- badges `Disponible` / `Déjà prêté` sur chaque fiche ;
- bouton `Emprunter` / `Retourner` sur chaque livre ;
- onglet `Mes Emprunts` connecté à `user-loans-service` ;
- historique des emprunts par utilisateur ;
- formulaire de création d'emprunt guidé.

Variables d'environnement :

```
VITE_BOOKS_API_URL=http://localhost:3001
VITE_LOANS_API_URL=http://localhost:3002
```

10. Conclusion

Ce projet nous a permis de mettre en pratique l'ensemble des compétences abordées en cours : architecture microservices, conteneurisation, orchestration Kubernetes et développement fullstack.

Les principaux acquis techniques sont :

- la conception d'une API REST propre avec gestion des codes d'erreur métier ;

— la mise en place d'une communication inter-services fiable via DNS interne Kubernetes ;
— la sécurisation des credentials avec les Secrets et le principe du moindre privilège RBAC ;
— la garantie de persistance des données avec les PersistentVolumeClaims ;
— la maîtrise du cycle complet : code → Docker → Docker Hub → Kubernetes.

L'application finale permet de gérer un catalogue de livres, créer des emprunts avec contrôle de disponibilité, enregistrer les retours et visualiser l'état du système dans une interface web unifiée – le tout déployé sur un cluster Kubernetes local.